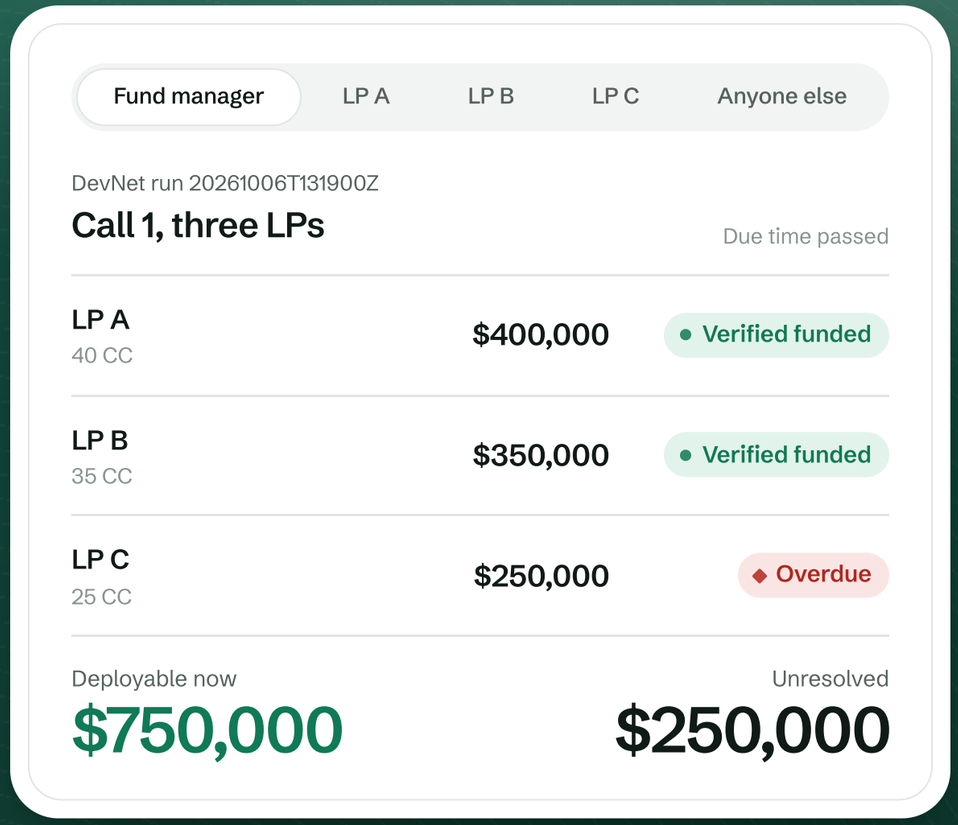
draw

Capital calls only the right investor can read.

DRAW runs a fund's capital call on Canton. Each LP gets a private notice and pays it with a Canton token. The fund counts money as deployable only after a separate read-back step confirms it arrived.

Live on DevNet: draw.timjosh507.workers.dev



DevNet, HackCanton shared node. DRAW's backend login signs for every demo seat. Settles in Canton Coin from the DevNet faucet. Dollar figures are the scenario.

Today a capital call is a notice, a wire and a spreadsheet.

Payments get matched in batches, from bank exports.

One fund software vendor’s service terms: “Client must provide a .csv file of recent bank transactions” before it can reconcile contributions, updated “up to three times per capital call.”

Juniper Square, Description of Services, rev. May 17, 2023

Partial, split and wrong-name payments are worked by hand.

Capital call tools reconcile each contribution against candidate bank transactions, and funds treat money as in only once it has cleared and been reconciled.

Product docs and ops guides, sources 9 to 11 in docs/PROBLEM_EVIDENCE.md

Fake notices with swapped payment details are a known fraud.

Funds fight it with callbacks, dual approval and faster reconciliation. All of it sits outside the payment itself.

FTI Consulting, SVB, Luxembourg FIU, sources 6 to 8

> A wide range of scams using fake capital call or drawdown notices, falsified invoices or fraudulent loan repayment instructions has been detected.

Luxembourg Financial Intelligence Unit
Trend alert #2, Business Email Compromise targeting the investment sector, November 2025

Built for the person who has to say what’s actually been funded.

Fund COO or operations lead

at an emerging fund manager

Works at	An emerging manager. In most small funds, operations issues the call itself
Owns	The notices, matching wires to LPs, following up late payers, reporting funded versus outstanding
Uses today	A spreadsheet or admin system, email, wires and a bank export

The question DRAW answers for them

How much of this call is deployable right now, and what’s still unresolved?

Or the outsourced administrator who runs the call for them

An administrator runs calls for several emerging managers at once, so one relationship reaches several funds.

The LP on the other side

Gets a notice no other LP can read, and a clear answer on whether its payment counted.

Who pays • Hypothesis

The manager, for a deployable figure it can trust. The administrator, for less matching work across its clients.

Issue once. Every LP pays its own notice.

One call, one ledger. The money moving and the notice turning funded happen in the same transaction.

01 — Fund manager

Issue

One transaction creates a private notice for each LP. Other LPs and outsiders can’t read it, even by contract id.

A contract signed by the fund and that LP

02 — LP

Pay

The LP locks the exact amount in a Token Standard allocation that carries the notice’s reference. Nothing has moved yet.

A CIP-0056 allocation to the fund’s party

03 — Fund manager

Settle

One transaction checks reference, payer, amount and asset, moves the coin and marks the notice funded. Anything else is refused.

The allocation is consumed, so it can’t pay twice

04 — Read-back

Verify

A separate step reads the settlement back and runs 14 checks. Only then does the money count as deployable.

Read-only login on LocalNet, a login of its own in the DevNet run

◆ Overdue

An LP that doesn’t pay stays unresolved. Before the due time the ledger refuses to mark it overdue. After it, the fund can. The fund can also send a notice to review, or waive an overdue or reviewed one, each with a note.

DevNet

It runs on DevNet today.

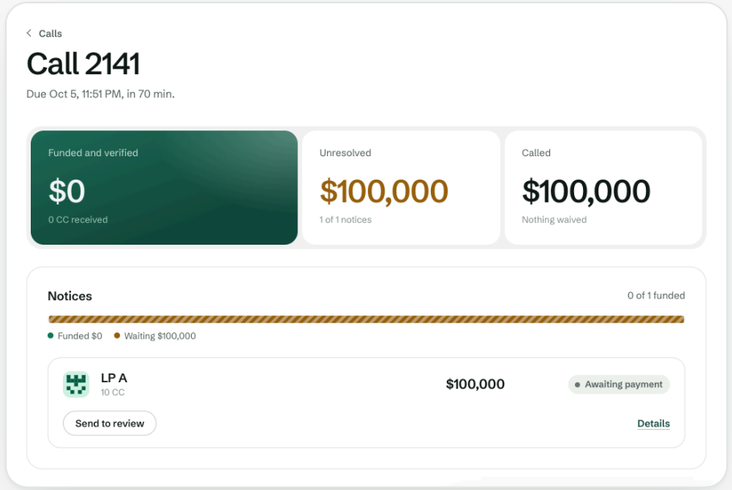
The deployed app in a real browser, against the HackCanton node, where its latest journey passed 10 of 10 steps.

> DevNet, HackCanton shared node. DRAW's backend login signs for every demo seat.

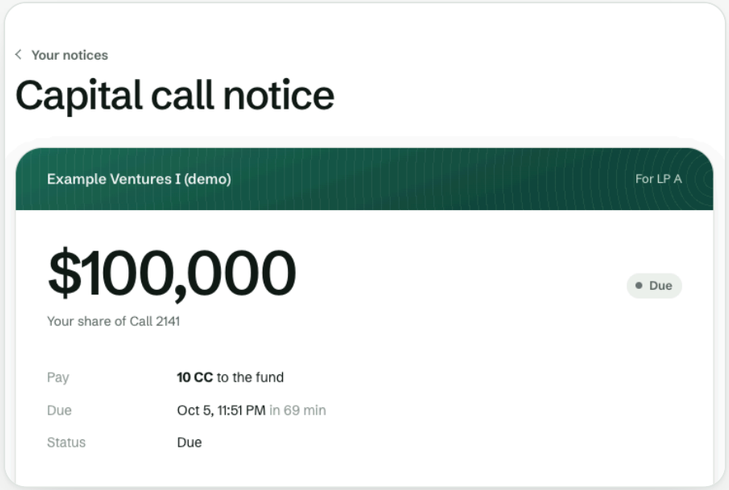
> Settles in Canton Coin from the DevNet faucet. Dollar figures are the scenario.

draw.timjosh507.workers.dev

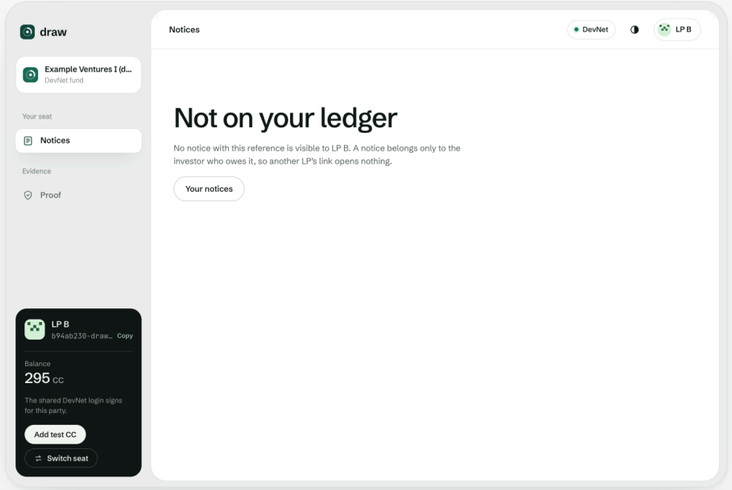
Have a HackCanton login? An LP can join as itself, read its notices with its own token, and pay with Accept in its own Canton Coin wallet. DRAW never gets the token. Every run used the team's own login as the LP.



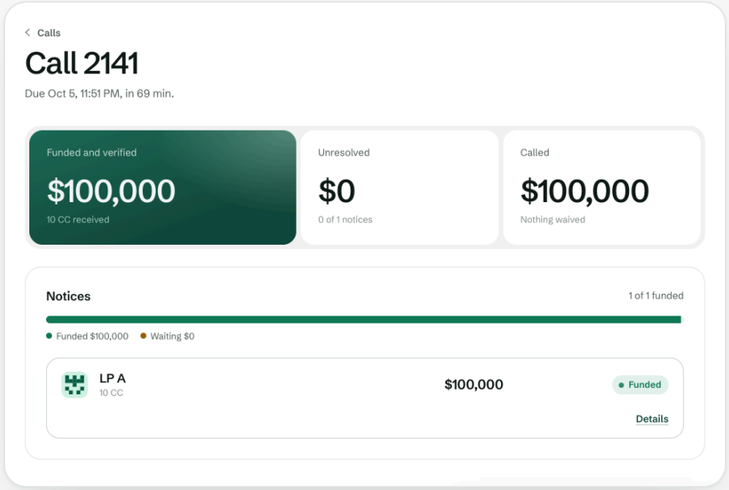
1 The fund manager issues Call 2141. LP A owes $100,000, paid as 10 CC, and all of it is unresolved.



2 LP A opens its own notice for Call 2141. Paying locks the 10 CC for the fund, and nothing moves yet.



3 LP B opens LP A's notice link. The ledger returns nothing.



4 The fund settles and the read-back step confirms it: $100,000 funded and verified, nothing unresolved.

draw

Screens from `evidence/devnet/browser/web-journey-20261005214111`, the 2026-10-05 UI, when one hackathon login signed for every seat. 1, 2 and 4 are cropped. Latest journey: `web-journey-20261006132858`. The live site reads each settlement back with the backend login.

DevNet

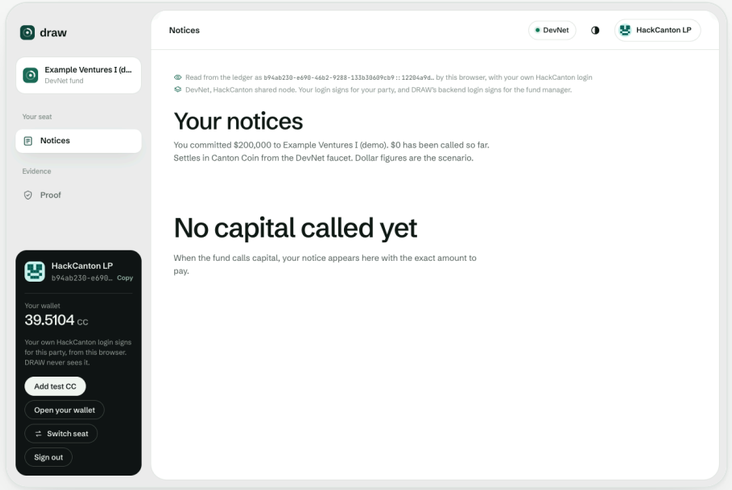
An LP pays from its own Canton wallet.

Each capital call also reaches the LP as a CIP-0056 allocation request. The node's Canton Coin wallet lists it, and Accept pays it.

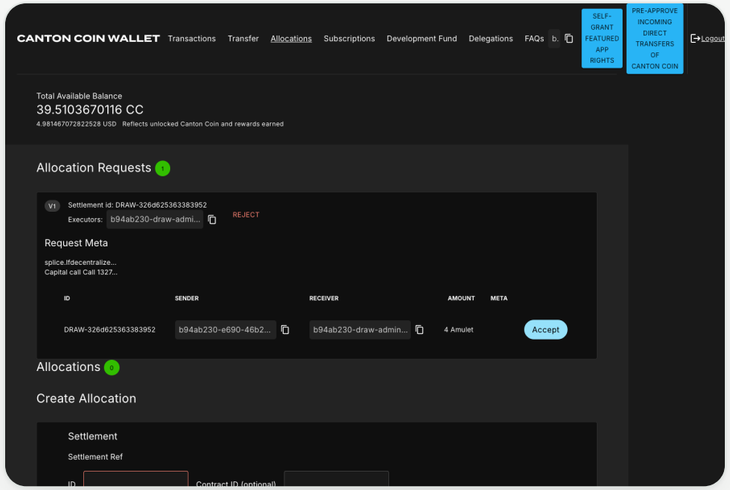
> DevNet, HackCanton shared node. The LP signs in with its own HackCanton login. The fund manager seat uses DRAW's backend login.

> DRAW's server learns only the LP's party id. Every run used the team's own login as the LP.

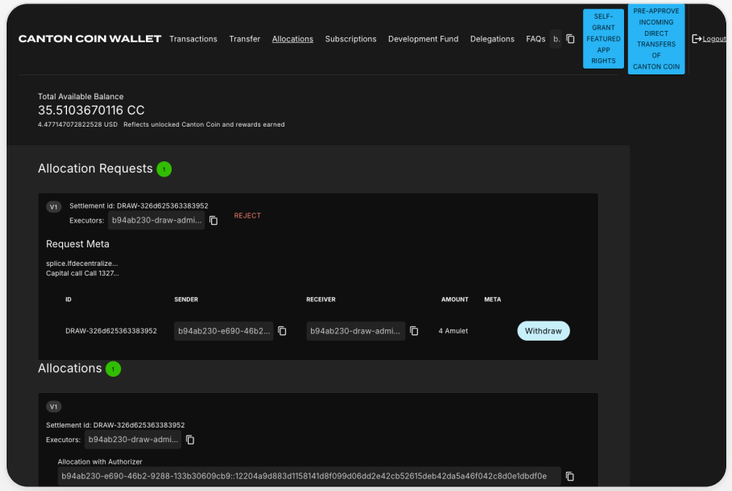
9 of 9 steps on the live site, from sign-in to verified funded. Paying from DRAW instead of the wallet passed 7 of 7. In the newest runs the server held nothing that could act for the LP's party.



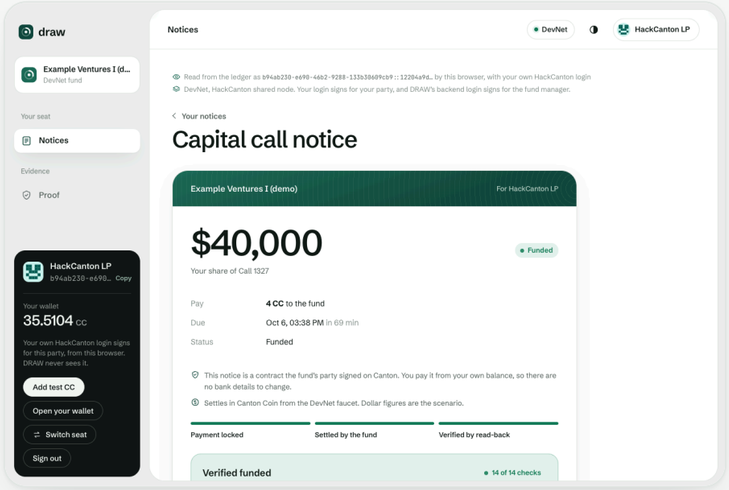
1 The LP signs in with its own HackCanton login and joins. Its browser reads its notices with its own token.



2 The fund calls 4 CC. The LP's Canton Coin wallet lists the call as an allocation request.



3 The LP presses Accept. Its wallet locks 4 CC for the fund, and nothing moves yet.



4 The fund settles and the read-back step confirms it. The LP's notice shows verified funded.

Screens from `evidence/devnet/browser/web-hackcanton-wallet-20261006132730`, with the server on the backend client. 2 and 3 show the top of the wallet page. Paying from DRAW instead: `web-hackcanton-20261006132613`.

Same call, same five attacks. A database let 4 through. DRAW let none through.

Attack	Database with a paid flag	DRAW on Canton
Mark an unpaid LP funded	◆ Got through	● Stopped
Read another LP’s notice	◆ Got through	● Stopped
Reuse one payment for another notice	◆ Got through	● Stopped
Count a payment row as money	◆ Got through	● Stopped
LP changes its own notice	● Stopped	● Stopped
Attacks that got through	**4 of 5**	**0 of 5**

Privacy sits below the app

With a login per party, as on LocalNet, LP A’s own login never gets LP B’s notice back, even by contract id. A missing filter in our code can’t change that.

Funded follows the money

The notice turns funded in the same transaction that moves the coin, and that transfer needs the LP’s prior consent.

A payment can’t be reused

A settled allocation is consumed. The same payment can’t fund a second notice.

What it doesn’t hide: the fund sees every LP by design, the node operator that hosts every party can read them all, and the Canton Coin leg shows up in Scan under a random reference.

`evidence/local/ablation/20261002T193830Z`, LocalNet. The database attacker has write access, as an operator or a compromised app server would.

Every number here comes from a script-written file.

DevNet

82/82

checks on the HackCanton node

Two LPs funded in test Canton Coin, bad payments refused, the unpaid LP overdue. The read-back's own login was refused as the fund. 1 check skipped: one backend login signs for every seat.

LocalNet

76/76

checks, each party holding its own key

The node can't act for a party without that party's signature, and another party's key is refused.

LocalNet

100/100

generated capital calls

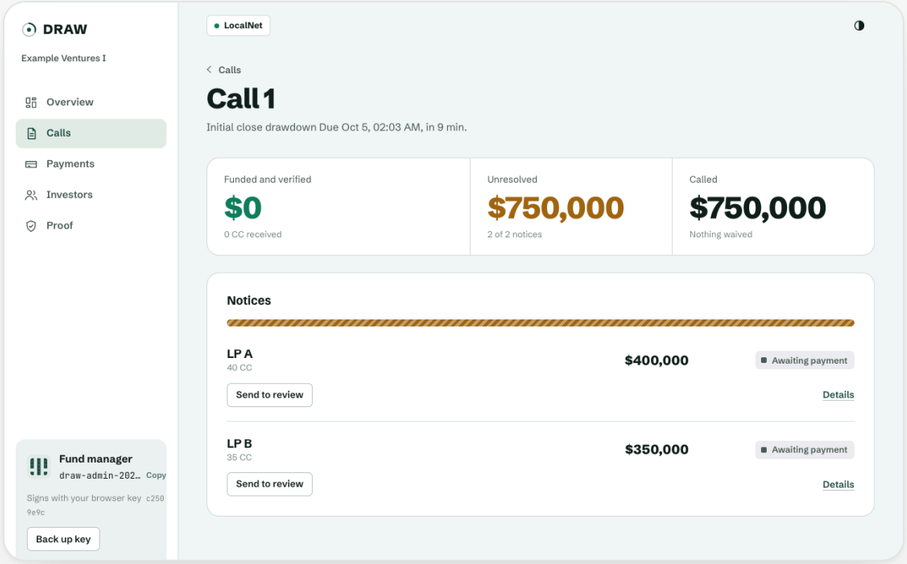
191 notices. 0 false fundings, 0 privacy violations, 0 replays, 0 total mismatches.

Daml Script

15/15

deliberately broken guards caught

Drop the amount, sender, reference, executor, instrument or deadline check and a test fails.



LocalNet. Each party signs with its own key.

6.8 s

DevNet

Settlement record time to LP A's read-back receipt, 4.5 s for LP B, in the shared-login run. The verifier's clock against the ledger's

1,047 ms

LocalNet

Payment locked to verified funded. One machine, scripted fund manager, not production latency

Runs: DevNet `20261006T131900Z` on package 0.3.0, the 6.8 s from DevNet `20261006T091531Z`, custody `20261004T183146Z`, campaign `20261002T193724Z`, mutation `20261006T081625Z`, workload `20261002T185353Z`, screen from browser keys `20261005005304`, cropped. How to check each one: docs/PROOF.md.

What’s on file, and what we measure next.

On file

Public evidence

12 sources

- An LP industry body, three law firms, two banks, a regulator, an incident responder and four vendors.
- 2 cut against us: LP defaults are rare. So we lead with late versus funded, and leave chasing defaulters out.
- Test evidence on DevNet and LocalNet, slide 8.

In progress

Conversations with people who run calls

- Goal: 3 to 5 COOs or ops leads at emerging managers, fund administrators or LPs, 20 minutes each.
- We record role, fund size band, LP count, calls per year, hours per call, the last incident and whether they’d shadow one call.

To fill after the interviews: number held (0 as of Oct 6), roles, what they said about hours per call and the last incident, and who’d shadow a call.

Planned

Usability tests on the live DevNet site

- Goal: 2 to 3 people, 15 minutes each.
- Tasks: issue a call, pay as LP A, settle as the fund, check that LP B can’t open LP A’s notice.
- We measure completion, time per task and where people got stuck.

To fill after the tests: completed N of N, median time per task, the top point of confusion.

In a shadow pilot we’d measure the gap between DRAW’s unresolved list and the fund’s own spreadsheet, the admin’s time per day in each, and any LP confusion about what it sees.

Go to market through the people who run the call.

Who first

One outsourced fund administrator serving several emerging managers, because one relationship reaches several funds. Then the COO or ops lead at emerging managers that run their own calls.

How we reach them

Direct outreach to fund ops people and administrators, introductions from people who run funds, and Canton ecosystem events. The interviews on slide 9 are the first pipeline.

Pricing

Hypothesis

A per-fund subscription for managers, and per-call pricing for administrators who serve many small funds. We haven't had a pricing conversation yet. Each interview asks what the fund pays for administration today.

Pilot, in three stages

Stage 1

Shadow one real call

No money moves. DRAW runs next to the fund's spreadsheet until the call closes. It needs commitment amounts and about an hour a day from one ops person.

Stage 2

A small test settlement

2 or 3 LPs pay a small test amount on DevNet or TestNet from their own parties. The real call stays on the bank rail.

Stage 3

What a real pilot needs

A settlement asset the fund and LPs accept, LPs on validators they trust, LP sign-in that gives our server no LP credentials, export to the admin system and legal review.

docs/PILOT_PLAN.md, docs/BUSINESS_BRIEF.md. Nobody has agreed to a pilot yet, and no customer or partner is implied.

Next: one real call in shadow, then TestNet.

Now

DevNet and LocalNet

The full call on the shared HackCanton node, and LP sign-in with a HackCanton login. A key per party on LocalNet.

Next

Shadow pilot

One real call alongside a fund or administrator. On DevNet, a read-back login that only reads, and a second person signing in.

Then

TestNet with USDCx

Needs Tokenomics Committee approval, an onboarding secret from a Super Validator sponsor and an allowlisted IP.

After a pilot

MainNet

LPs on validators they trust, export to the administrator's system, then Featured App review.

The ask

1. Introduce us to an emerging manager's COO or ops lead, or a fund administrator, who'd shadow one real call with no money moving.
2. Help us onto TestNet: a Super Validator sponsor and a Tokenomics Committee review.

> Uses the Token Standard allocation interface, which the USDCx registry supports. Not run with USDCx.

Live	draw.timjosh507.workers.dev
Repo	Public GitHub link to add
Video	Video link to add